# Triple and quadruple large primes, "outside the box" probes, and the byte-counter decision

**Status: exploratory, not preregistered, no leaderboard or benchmarks row.** The goal (index calculus near rho parity on a single target) is **not met**. What this study found is where the overhead comes from and the one regime in which it disappears.

## 1. Triple / quadruple large primes (analysis plus in-repo measurements)

A k-large-prime scheme lets a relation contain k unknown "large" group elements; relations are combined through a graph or hypergraph whose cycles cancel the large primes. That reduces how many relations are needed for rank (a hypergraph cycle count, linear in the number of large primes). It does not change what a relation costs. A relation with k large primes drawn from a set of size l is a k-summand coincidence among group elements, and in a prime-order elliptic-curve group there is no homomorphic coordinate, so partial-collision tricks (Wagner's k-tree) do not apply. Detection is a generic k-sum search, so the whole scheme is a generic-group algorithm and Shoup's bound (about sqrt(r) group operations) applies.

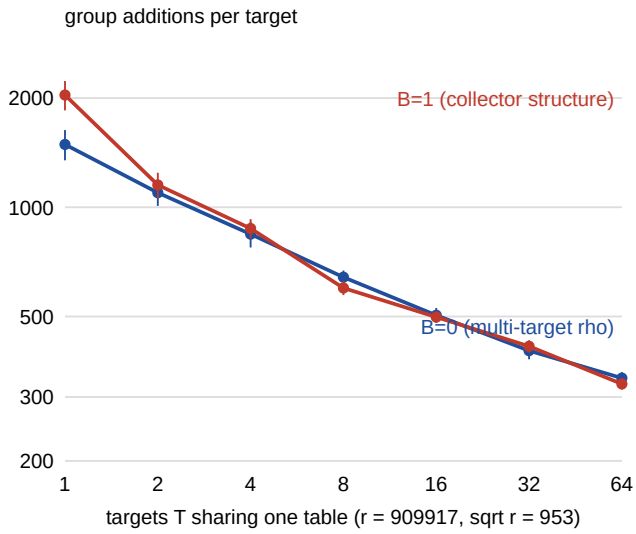
| **summands k** | **generic best exponent k/(2k-1)** | **measured in-repo (lead atlas, ic.ksum, 40-bit prime curve, w=0)** |
|---|---|---|
| 2 (double LP / pair table) | 0.667 | not re-quoted here (atlas: pair-table pipelines grow faster than rho) |
| 3 (triple LP) | 0.600 | not re-quoted here (atlas: mitm decomposition exponents 0.60-0.75 at 12-32 bits, rho 0.50) |
| 4 (quadruple LP) | 0.571 | 0.565 [0.560, 0.571]; S = 11.56 |
| 5 | 0.556 | 0.547 [0.543, 0.554]; S = 8.51, 9.7x rho-neg |
| rho | 0.500 | 0.50 [0.48, 0.52] |

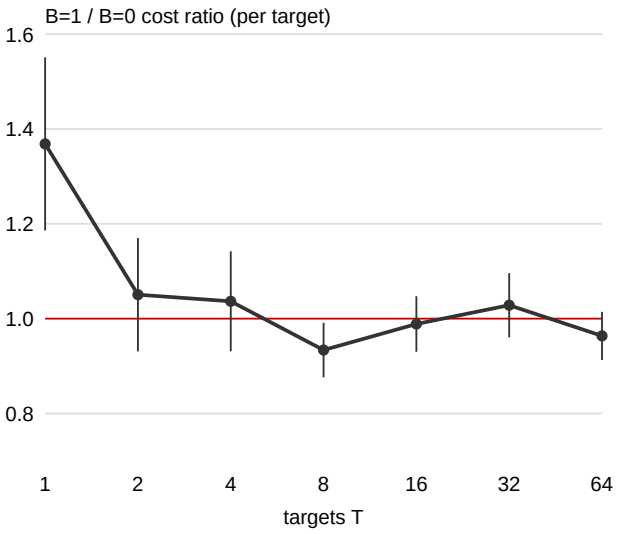
The exponent only approaches 1/2 from above as k grows, with memory approaching sqrt(r). Triple and quadruple large primes were therefore already measured here as the k-sum tracks, and are worse than rho at every size tested. The numbers in the right-hand column are quoted from the crypto-autoresearcher lead atlas (`LEAD-ICP-generic-table-decomposition`), not re-run. Constants: Shoup's bound is an asymptotic statement, so what it rules out is a better exponent; any generic variant could at most gain a constant over rho, and none is close to parity today (res1eq = 1.60x).

## 2. Where the overhead actually comes from, and the regime where it vanishes

The residual collector is a rho-style collision search that needs B+1 independent relations for B unknown factor-base logs plus the target. For one target and B=1 the exact expected cost is Gamma(2.5)/(Gamma(2)Gamma(1.5)) = 1.5 times a single collision, which is the floor res1eq (1.60x rho) sits near. That overhead is an artifact of having one target. With T targets sharing one table, T unknown logs need about T relations either way, so adding B extra unknowns changes the count from T to T+B and the relative overhead falls to about B/(2T).

**Prototype (standalone Python, 20-bit prime-order curve, r = 909917, 12 trials per cell, every recovered log checked against the planted value):** walks step only by multiples of G, so trails of different targets can merge; every collision is one linear equation over GF(r); counts are group additions.




| T | adds / target, B=0 | adds / target, B=1 | B=1 / B=0 |
|---|---|---|---|
| 1 | 1490 +- 143 | 2038 +- 189 | 1.37 +- 0.18 |
| 4 | 843 +- 69 | 874 +- 53 | 1.04 +- 0.11 |
| 16 | 504 +- 24 | 498 +- 18 | 0.99 +- 0.06 |
| 64 | 338 +- 13 | 326 +- 12 | 0.96 +- 0.05 |

The B=1 overhead is 1.37x at T=1 (consistent with the 1.5 prediction within the noise) and indistinguishable from 1 for T of 8 or more. Per-target cost at T=64 is 4.4x below T=1 (the ideal sqrt(T) would be 8x; the loss is the connectivity cost of the equation graph, roughly sqrt(ln T)).

**Honest reading:** this is multi-target rho (Kuhn-Struik; Bernstein-Lange), reproduced. The B=0 curve *is* a multi-target rho. It beats a single-target rho by up to 4.4x here, but a fair reference for a T-target batch is multi-target rho, and against that the collector's ratio goes to 1 (parity), not below. It is not index calculus beating rho, and a batch changes the unit of S, so by the repository's success criterion it is a separate track, not an improvement of the single-target baseline. Caveats: one small curve, no negation map, no distinguished points (the whole table is stored, memory not charged), start-point setup not counted, Python prototype outside ecbench.

## 3. Other probes

| idea | status | result |
|---|---|---|
| Coordinate-leakage battery: 15 bit/Legendre/size features of x([k]G), y vs 18 bit/residue/size targets of k, p36 curve, N = 60,000, 272 pairs (script `scripts/leak.py`) | run | Null. Largest \|z\| = 3.26 against a Bonferroni threshold of 3.74 and a null expectation of about 3.35. Planted positive controls: a 2% match-probability bias reads \|z\| = 8.9, so correlations of about 1.5% or more would have been seen. |
| Small-x decomposition by Coppersmith / lattice small roots on S_3 (algebraic, p-adic-free) | estimate, not run | Even under the optimistic bound X*Y at most p^(1/2) for the degree-(2,2) polynomial, base size p^(1/4) gives decomposition probability p^(-1/2) per target, so about p^(1/2) lattice calls per relation and p^(3/4) in total, each far more than one group addition. Worse than rho. The atlas lists this lead without a run. |
| Isogeny to a curve with more automorphisms (j = 0 or 1728) | analysis | Would speed up rho by sqrt(\|Aut\|/2) on that curve, but only exists in the class for special traces, and it helps the matched rho equally. Not a relation source. |
| Endomorphism orbits (GLV) | analysis | The equivalence class under a non-unit endomorphism has size ord(lambda), about r for generic lambda, so canonicalisation costs more than the walk. Only units (size 2, 4, 6) are usable, and eq already takes size 2. |

| | | |
|---|---|---|
| Pairing / MOV transfer | analysis | Needs a small embedding degree; for a random prime-order curve the degree is huge. Not applicable to a generic target. |

## 4. The byte counter: assessment and decision

The question: should `residual_table_bytes_uncharged` count as an operation? **Current state:** AGENTS.md section 5a (owner, 2026-11-01) already adopted option B: the band prices operations, not bytes. `NON_OPERATION_COUNTERS` in `src/cryptanalysis/ecbench/band.rs` lists the byte counters, which stay listed with their counts but are not charged *w*; audit in `research/ic_byte_counter_pricing_20261101/`. My earlier "owner decision outstanding" notes were stale.

**Why charging a byte at w GAE is wrong:** a GAE is one group addition, about 86,000 bit operations on these curves (35.4e9 bit operations over 410,591 additions in a res1eq run). Charging one GAE per byte makes 12.2 MB of baseline table cost 12 million GAE, about sixteen times the arm's whole group work (766k GAE), which is what produced the 17-58x figures before option B. The time spent using the table is already counted by its probes (`lookups`) and inserts.

**Why it still counts as an operation, at its true price:** writing the table is a word store. The repository already prices the factor-base vector that way in the `*_full` methods (one word store per eight bytes, `full_price.rs`). On the same basis (about 64 bit operations per word store, my estimate) the residual table is: res1eq 34 KB, about 4,300 words, about 3 GAE out of about 385,000 (0.001%); the baseline residual arm (res2aa) 12.2 MB per run (978 MB over 80 runs), about 1,140 GAE out of 766,000 (0.15%). No ranking changes under any price from zero up to this one.

**Measured effect on my own reported numbers:** pricing bytes at one GAE per byte would move res1eq at w=1 from 2.69x to 2.76x rho (+2.6%). The figures I reported (1.60x, 1.80x, 2.70x for w = 0, 0.1, 1) are exclusive of bytes and consistent with option B.

**Memory is still a real cost, and belongs on its own axis.** Peak table: rho about 440 distinguished points (about 10 KB if 24 bytes each, an assumption); res1eq 34 KB (about 3x that); res2aa 12.2 MB (about 1,200x). The Iso Lab L2 replay measured the real-world price directly: res2aa runs at 1.463x rho's nanoseconds per counted operation (CI [1.442, 1.484]), a cost of memory showing up as time per operation rather than as extra operations.

**Decision:** (1) Keep option B; do not change the band or any emitter. (2) Treat byte counters as operations only at word-store price, immaterial here, so no price is added to the band. (3) Report peak table bytes and ns/GAE as a separate memory axis next to every IC-versus-rho row. This needs no code change; whether to add the word-store line to the `*_full` pricing for the residual counter is a small follow-up I did not make.

## 5. Decisions on direction

1. Triple and quadruple large primes: closed by the generic bound and by the repository's own k-sum measurements. I will not build them.
2. Single-target index calculus on a generic prime-order curve cannot reach parity with the methods known; the floor is the exact 1.5x collision overhead plus whatever constant a perfect generic algorithm still holds over rho.
3. The one place the gap closes is the batch regime. I recommend recording it as its own track: a T-target shared-table collector measured in ecbench against *multi-target rho* as the matched reference, with distinguished points and memory reported, before any claim. That needs an ecbench method and a matched reference; it is not built yet.

Scripts and logs: `scripts/mt.py`, `scripts/leak.py`, `logs/`. Not covered: negation, distinguished points, larger curves, hardware effects.